

Chapter recap

Last updated February 2024

Great job!

Remember that all the TypeScript types you're writing will disappear from your code. They are only useful at compile time to prevent runtime errors with your JavaScript code.

You can check this out one more time using the [official TS Playground](#).

Try pasting the code below:

```
function formatValue(value: string | number) {  
  if (typeof value === 'string') {  
    return value.toLowerCase();  
  } else {  
    return value.toFixed(2);  
  }  
}
```

Notice how the `: string | number` disappears from the output code. On the other hand, `typeof` is a JavaScript operator that runs at runtime, thus it remains in the outputted code.

If we zoom out a little bit, TypeScript is already helping us catch some runtime bugs by making sure we only call methods that are available depending on the type of the variable. We also benefit from a rich autocomplete experience in our editor.

Without TypeScript, you may have encountered error messages such as:

Cannot call `.toLowerCase()` on `count` (when `count` would have been a number).

As a cost for these benefits, we have to get familiar with TypeScript, its syntax, and its error messages. But, it's totally worth it!

Chapter Recap

- A **union type** is a new type that we create by specifying that any of the two (or more) specified types can be accepted.
- `string | number` is a union type between `string` and `number`. It means the type is allowed to be a `string` or a `number`.
- When you have a union type, you can only call a method if it's valid on every type provided in the union type.
- Narrowing is when TypeScript can deduce a more specific type based on your code.
- TypeScript types are not available at runtime. However, you can use the JavaScript `typeof` operator to create different behaviors based on different runtime types.

Was this helpful?

